Webprogrammierung Hausarbeit

Niklas Klein

07.12.2024

Inhaltsverzeichnis

Hausarbeit als Prüfungsleistung WiSe 2024/2025 **2**
Abgabefrist . . . 2
Aufgabe . . . 2
Bewertungskriterien . . . 2
Funktionale Anforderungen (60 Punkte) . . . 2
1. Kern-CRUD-Operationen (20 Punkte) . . . 2
2. Komplexe Beziehungen (15 Punkte) . . . 3
3. Such- und Filterfunktionalität (15 Punkte) . . . 3
4. Eigenes Feature (10 Punkte) . . . 4
Nicht-funktionale Anforderungen (20 Punkte) . . . 4
Präsentation der Ergebnisse (20 Punkte) . . . 5
Bewertungsrichtlinien: . . . 6
Bildung der Note aus den Punkten . . . 6
Hinweise zur Umsetzung und Bewertung . . . 7
Zusammenarbeit und individuelle Beiträge . . . 7
Abgabe . . . 7
Unterstützung und Ressourcen . . . 7
Ich wünsche euch viel Erfolg! . . . 8

Hausarbeit als Prüfungsleistung WiSe 2024/2025

Achtung: Dies ist eine Gruppenhausarbeit.

Bitte lest den gesamten Text aufmerksam durch, bevor ihr mit der Aufgabe beginnt. Bei Fragen wendet euch bitte im Discord oder im Labor an mich.

Abgabefrist

Abgabe bis zum 05.01.2024 um 23:59:59.99999

Aufgabe

Entwickelt eine Film-Datenbank-Anwendung mit Python und Django. Die Anwendung soll es Nutzern ermöglichen, Informationen über Filme, Schauspieler und Regisseure zu verwalten. Zusätzlich sollen Nutzer Filme durchsuchen und filtern können. Schließlich soll ein eigenes, von euch erdachtes Feature implementiert werden, das die Anwendung erweitert und die Benutzererfahrung verbessert.

Bewertungskriterien

- **60 %** der Note ergeben sich aus der **Erfüllung der funktionalen Anforderungen**.
- **20 %** der Note ergeben sich aus der **Einhaltung der nicht-funktionalen Anforderungen**.
- **20 %** der Note ergeben sich aus der **Präsentation der Ergebnisse**.

Funktionale Anforderungen (60 Punkte)

1. Kern-CRUD-Operationen (20 Punkte)

Implementiert die vollständigen CRUD-Funktionen (Erstellen, Anzeigen, Aktualisieren, Löschen) für die folgenden Modelle:

- **Film**
  - Felder: Titel, Veröffentlichungsdatum, Genre, Beschreibung.
- **Schauspieler**
  - Felder: Vorname, Nachname, Geburtsdatum, Biografie.
- **Regisseur**

  - Felder: Vorname, Nachname, Geburtsdatum, Biografie.

- **Bewertung**
  - Felder: Film (ForeignKey), Rezensent (Pseudonym), Bewertung (z.B. 1 bis 5 Sterne), Kommentar, Datum.

Hinweise:

- Die CRUD-Funktionen sollen über benutzerfreundliche Webseiten zugänglich sein (nicht über das Django-Admin-Interface).
- Stellt sicher, dass Nutzer Bewertungen zu Filmen hinzufügen können.

2. Komplexe Beziehungen (15 Punkte)

- Modelliert die Beziehungen zwischen Filmen, Schauspielern und Regisseuren korrekt:
  - **Viele-zu-Viele-Beziehung** zwischen Filmen und Schauspielern.
    * Ein Film kann mehrere Schauspieler haben.
    * Ein Schauspieler kann in mehreren Filmen mitspielen.
  - **Eins-zu-Viele-Beziehung** zwischen Regisseuren und Filmen.
    * Ein Film hat einen Regisseur.
    * Ein Regisseur kann mehrere Filme gemacht haben.
- Erstellt Ansichten, die diese Beziehungen darstellen:
  - Seite, die alle Filme eines bestimmten Schauspielers anzeigt.
  - Seite, die alle Schauspieler in einem bestimmten Film anzeigt.
  - Seite, die alle Filme eines Regisseurs anzeigt.
- Ermöglicht die Navigation zwischen diesen Ansichten (z.B. von der Filmseite zu den Schauspielern und zum Regisseur).

3. Such- und Filterfunktionalität (15 Punkte)

- Implementiert eine **Suchfunktion**, mit der Nutzer nach Filmen suchen können basierend auf:
  - Titel
  - Genre
  - Schauspieler
  - Regisseur

- Implementiert **Filteroptionen**, um Filme einzugrenzen:
  - Veröffentlichungsjahr (Zeitraum)
  - Durchschnittliche Bewertung
- **Kreative Erweiterung:** Ihr könnt zusätzliche Filter hinzufügen, z.B. nach Filmlänge, Sprache, etc.

4. Eigenes Feature (10 Punkte)

- **Anforderung:** Implementiert mindestens ein einzigartiges Feature, das die Benutzererfahrung verbessert.
- **Beispiele:**
  - **Benutzer-Watchlist:**
    - Nutzer können Filme zu einer persönlichen Watchlist hinzufügen.
    - Anzeigen der Watchlist auf der Profilseite des Nutzers.
  - **Empfehlungssystem:**
    - Biete personalisierte Filmempfehlungen basierend auf bevorzugten Genres oder zuvor bewerteten Filmen an.
  - **Kommentarfunktion:**
    - Erlaube Nutzern, Kommentare zu Schauspielern zu hinterlassen.
    - Implementiere Diskussionsforen oder Kommentar-Threads.
- **Hinweis:** Beschreibt in eurer Präsentation, welches Feature ihr implementiert habt und warum ihr euch dafür entschieden habt.

Nicht-funktionale Anforderungen (20 Punkte)

1. **Verwendung von Python und Django** (Pflicht)
   - Die Anwendung muss mit Python und Django umgesetzt werden.
   - JavaScript kann optional für verbesserte Benutzeroberflächen verwendet werden, aber die Kernfunktionalität soll serverseitig in Django implementiert sein.
2. **Klassenbasierte Views** (2 Punkte)
   - Nutzt, wo sinnvoll, klassenbasierte Views (`Class-Based Views`).
3. **Fehlerbehandlung** (2 Punkte)

- Bei Aufrufen von URLs mit nicht vorhandenen IDs soll eine benutzerfreundliche 404-Fehlerseite angezeigt werden.

4. **Projektstruktur** (2 Punkte)
    - Organisiert euer Projekt in einer üblichen und sinnvollen Struktur.
    - Verwendet mindestens eine eigene Django-App (`python manage.py startapp ...`).
5. **Dokumentation und Kommentare** (2 Punkte)
    - Funktionen, Klassen und Module sollen mit aussagekräftigen Kommentaren und Docstrings versehen sein.
    - Verwendet den PEP 257 Style Guide für Docstrings.
6. **Templates** (2 Punkte)
    - Verwendet ein Basis-Template und nutze Template-Blöcke (`{% block content %}{% endblock %}`) für wiederkehrende Elemente.
7. **Abhängigkeiten** (2 Punkte)
    - Verwaltet eure Abhängigkeiten mit einer `requirements.txt`-Datei.
8. **Benutzeroberfläche** (2 Punkte)
    - Die Anwendung soll optisch ansprechend sein.
    - Ihr könnt CSS-Frameworks wie Bootstrap oder Tailwind CSS verwenden.
9. **Code-Qualität** (4 Punkte)
    - Vermeidet Code-Duplikationen.
    - Verwendet sinnvolle und aussagekräftige Variablen- und Funktionsnamen.
    - Entfernt ungenutzte Importe und unnötigen Code.
    - Haltet euch an den PEP 8 Style Guide für sauberen und lesbaren Code.

Präsentation der Ergebnisse (20 Punkte)

Stellt eure Anwendung im Labor in der ersten Januarwoche (06.01. bzw. 09.01.) vor (15-20 Minuten). **Alle Gruppenmitglieder** sollen an der Präsentation beteiligt sein. Die Präsentation kann auch über Discord erfolgen.

Bewertungsrichtlinien:

- **Einhaltung des Zeitrahmens** (5 Punkte)
  - Die Präsentation sollte zwischen 15 und 20 Minuten dauern.
- **Demonstration der implementierten Funktionen** (5 Punkte)
  - Zeigt die Kernfunktionen deiner Anwendung.
- **Erklärung der gewählten Lösungen und Herausforderungen** (5 Punkte)
  - Diskutiert eure Herangehensweise und eventuelle Probleme, die ihr gelöst habt.
- **Vorstellung des eigenen Features** (5 Punkte)
  - Präsentiert euer einzigartiges Feature und erläutert, warum ihr euch dafür entschieden habt.

Bildung der Note aus den Punkten

Prozent (Punkte)	Note
100 - 94 %	1,0 (sehr gut)
<94 - 89 %	1,3 (sehr gut -)
<89 - 84 %	1,7 (gut +)
<84 - 79 %	2,0 (gut)
<79 - 73 %	2,3 (gut -)
<73 - 68 %	2,7 (befriedigend +)
<68 - 63 %	3,0 (befriedigend)
<63 - 57 %	3,3 (befriedigend -)
<57 - 52 %	3,7 (ausreichend +)
<52 - 50 %	4,0 (ausreichend)
<50 %	5,0 (nicht bestanden)

Hinweise zur Umsetzung und Bewertung

Zusammenarbeit und individuelle Beiträge

- **Git und Versionskontrolle**
  - Nutzt Git zur Versionskontrolle eures Projekts.
  - Achtet auf aussagekräftige Commit-Nachrichten.
  - Die Git-Historie wird zur Bewertung der individuellen Beiträge herangezogen.
- **Individuelle Leistung**
  - Sollte die Leistung innerhalb der Gruppe ungleich verteilt sein, besteht Anspruch auf eine individuelle Benotung.
  - Individuelle Beiträge können anhand von Git-Commits, Beteiligung an der Präsentation und Rückmeldungen der Teammitglieder beurteilt werden.
- **Problemlösung innerhalb der Gruppe**
  - Versucht zunächst, eventuelle Probleme innerhalb der Gruppe einvernehmlich zu lösen.
  - Wenn keine Einigung erzielt werden kann, wendet euch frühzeitig an mich (niklas.klein@hs-flensburg.de).

Abgabe

- **Deadline einhalten**
  - Die Abgabe erfolgt bis zum 05.01.2024 um 23:59:59.99999.
  - Verspätete Abgaben können nicht berücksichtigt werden.
- **Abgabeformat**
  - Reicht die Abgabe über GitHub Classroom ein.
  - Verwendet eine `.gitignore`-Datei, um Umgebungsdateien (z.B. `.venv`) und unnötige Dateien vom Upload auszuschließen.

Unterstützung und Ressourcen

- **Fragen und Hilfe**
  - Bei Fragen oder Problemen nutzt den Discord-Server oder sprecht mich im Labor an.
  - Zögert nicht, Hilfe zu suchen, wenn ihr nicht weiterkommst.

- **Ressourcen**
  - Offizielle Django-Dokumentation: https://docs.djangoproject.com/
  - PEP 8 Style Guide: https://www.python.org/dev/peps/pep-0008/
  - PEP 257 Docstring Conventions: https://www.python.org/dev/peps/pep-0257/

Ich wünsche euch viel Erfolg!

Nutzt diese Aufgabe als Gelegenheit, eure Fähigkeiten in Python und Django zu vertiefen und kreativ zu sein. Ich freue mich auf eure innovativen und gut durchdachten Lösungen!